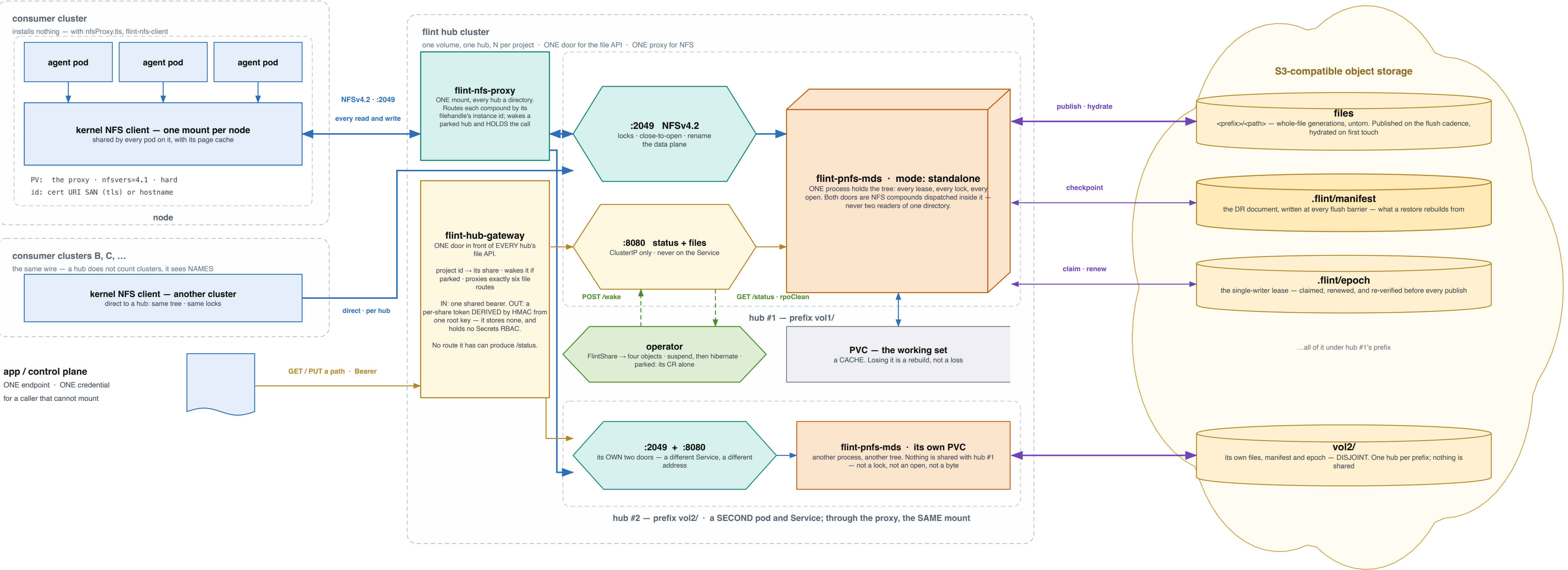


Data flow

One process holds each tree, so N pods in N clusters see one filesystem. One door fronts every hub's file API, and one proxy fronts every hub's mount.



BOTH DOORS CAN BE FRONTED, each by its own component. The FILE API: flint-hub-gateway resolves a project id to its share, wakes it if it is parked, and proxies six file routes — three thousand shares through ONE endpoint and ONE credential, as forge's door fronts every repository. THE MOUNT (v1.57.0, nfsProxy.enabled): flint-nfs-proxy terminates NFSv4.1 sessions on one LoadBalancer :2049 and routes each compound to the hub named by the instance id every hub filehandle already carries, so one mount lists every workspace and hubs stay one per share. Its client table is persisted, so a proxy restart costs a client a new session, not its state. Without the proxy a consumer mounts each hub's own routable address (spec.service.advertiseAddress → status.address): a mount per hub.

AND THE GATEWAY EXISTS BECAUSE PER-SHARE EXPOSURE DOES NOT SCALE. A hub's file API answers at a HEADLESS in-cluster Service the operator deliberately never makes routable: 3000 ClusterIPs exhaust a GKE-default /20 at about 2048 shares — and once the allocator is dry, a new share's CONSUMER Service cannot be created either, so one tenant's share count would stop every other tenant mounting anything — while NetworkPolicy cannot be the guard (off by default, hubNamespaces defaults to [], some CNIs ignore every rule in silence). One gateway can be guarded; three thousand cannot.

ONE KEY, NOT THREE THOUSAND SECRETS. Inbound is one shared bearer, re-read every 10 s so rotation is a Secret edit with no restart. Outbound is per-share and DERIVED — HMAC-SHA256(root, endpoint : bucket : keyPrefix : version) — so the gateway stores no hub credential and has no Secrets RBAC at all; the alternative was `get secrets` in every tenant namespace, where those tenants' S3 keys live. The caller's own credential is never forwarded upstream, and no route the gateway has can produce /status.

ONE PROCESS IS THE COHERENCE AUTHORITY, and that is why each hub is one pod. Every lease, lock and open lives in it, so N pods across N clusters see one tree — held to one by Recreate, an exclusive flock and the RWO attach fence inside the cluster, and by the epoch cell in the bucket across clusters. A second hub on ONE prefix is split-brain: it loses the CAS and is fenced. Two hubs on two prefixes, as drawn, is the ordinary configuration.

A HUB DOES NOT COUNT CLUSTERS — IT SEES CLIENT NAMES. On a direct mount the NFSv4 client identity is the hostname and nothing else, so two clusters sharing a node name are ONE client, and RFC 8881 §18.35.5 requires the server to read the second as the first rebooting: the incumbent's locks and opens go, silently, correctly, on a false premise. Set hostname = <cluster>-<node>, or nfs.nfs4_unique_id. Through the proxy with nfsProxy.tls, the identity is the client certificate's URI SAN and sessions are bound to it.

THE DISK IS A CACHE AND THE BUCKET IS THE COPY, so the recovery point is the flush cadence. Cold files evict at a disk watermark and hydrate on first touch, and both doors SAY SO rather than hanging: NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After. A file request on a parked share WAKES it — measured at 11 s on kind — which is why the gateway watches the CR and never polls a hub: a poll counts as activity and would pin every share it touched awake. So does an NFS call through the proxy: a refused connection stamps chert.us/requested-at, and the proxy HOLDS the compound, retrying, for up to wakeHoldSecs (20 s) before it answers DELAY — 2.4–2.9 s from suspend, 5.7–7.3 s from hibernate (box, v1.57.0). A direct mount of a parked hub still hangs: nothing on its path can wake it.

IDLE BY DEFAULT BEHIND THE PROXY. nfsProxy.idleDefaults applies to every share without its own spec.idle: suspend after 3600 s, hibernate after 86400 s (dropped for a share with no bucket); spec.idle: {} opts out. Without the proxy the ladder stays opt-in, since nothing would wake a direct consumer. Hibernation verifies the bucket can rebuild the volume and that no client holds state, then deletes the PVC; behind the proxy the Deployment, Service and ConfigMap go too, so a parked share is its CR alone, and a wake re-renders them.

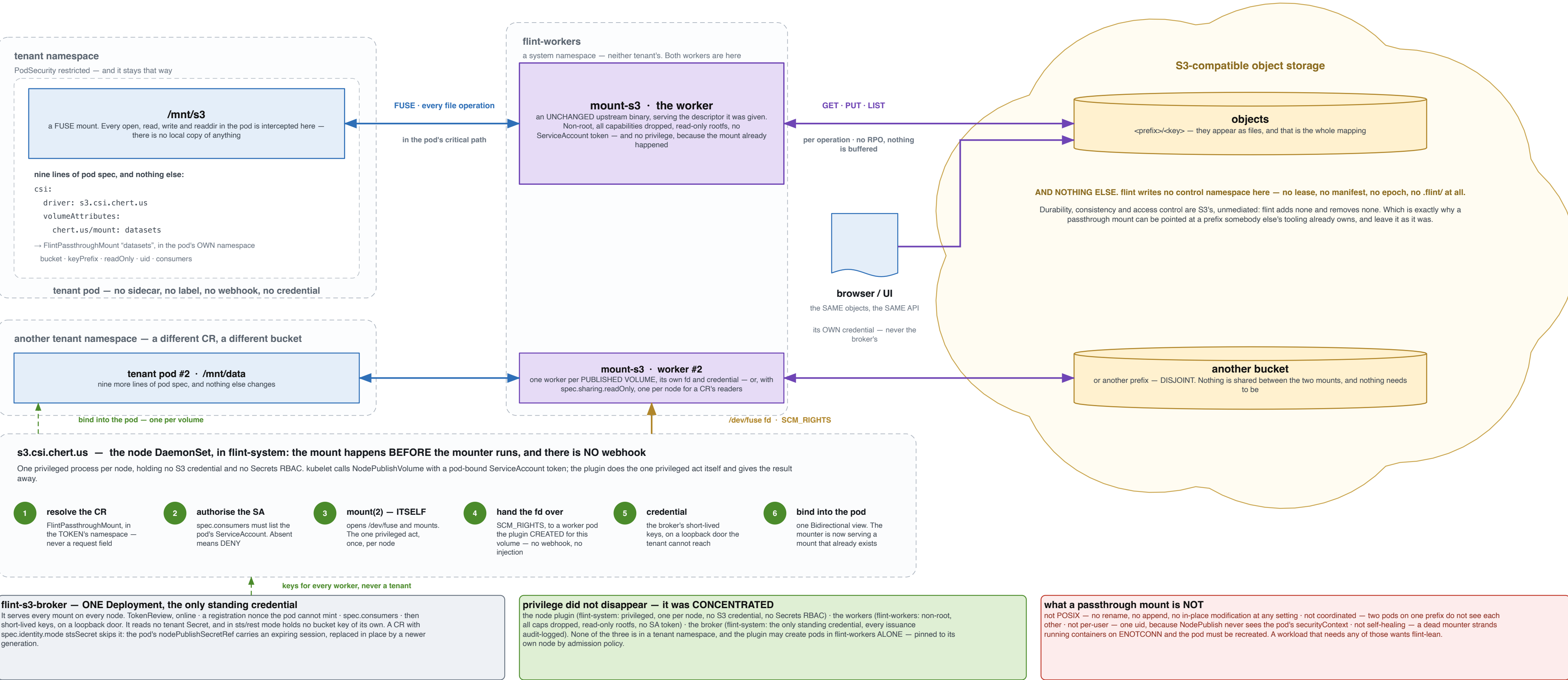
A HUB'S :2049 AUTHENTICATES NOBODY, so reachability is its boundary: kube-proxy SNATs a remote client to an address in the hub's own cluster before the packet arrives (1486 of 1486 connections on a three-cluster rig), so networkPolicy cannot draw it — use peering, security groups or a gateway. The PROXY can authenticate: with nfsProxy.tls a connection must open with the RFC 9289 AUTH_TLS probe and present a client certificate chained to clientCa (kernel >= 6.5 with tlsd; the flint-nfs-client chart keeps each node's pair current), and its identities rules decide who sees which workspace. Activity: the gateway sees every file-API call and the proxy every proxied compound, but a direct mount dials the hub itself, so its busy agents look idle to everything else and POST /wake has to be driven by whatever knows work is happening.

What the abbreviations mean

NFSv4.2 one protocol, one long-lived TCP flow, and a client already in every node's kernel	close-to-open NFS's coherence rule: what a writer closed, the next opener sees	AUTH_SYS the flavour port 2049 speaks: the client asserts its own uid, and nothing checks it	nconnect how many TCP connections one mount opens. Below 2 there is no trunking to spread
NFS4ERR_DELAY the server telling a caller to come back — how a cold read parks instead of hanging	epoch cell the object that says who the hub is. A second hub loses the CAS and is fenced	flush cadence how often closed generations publish. It is the recovery point, stated as a number	hydrate fetching an evicted file back from the bucket on first touch, into its own inode
evict · watermark cold files leave the PVC at a disk watermark — a marker is written before the truncate	.flint/manifest the DR document: the metadata checkpoint a restore rebuilds the tree from	rpoClean /status answering "can the bucket rebuild this volume right now?" — ask it before deleting a PVC	FlintShare the custom resource one lite volume is declared as. One volume, one hub, N of them per project
flint-nfs-proxy the door in front of every hub's :2049 — one address, one mount, every workspace a directory	instance id the hub's id inside every filehandle it issues: what the proxy routes a compound by	CR-only park a hibernated share behind the proxy: Deployment, Service and ConfigMap removed; a wake re-renders them	RPC-with-TLS RFC 9289: NFS over TLS, opened by an AUTH_TLS probe. The proxy can require a client certificate
flint-hub-gateway the door that fronts every hub's file API: one endpoint, one inbound credential	headless Service a Service with no ClusterIP — addressable inside the cluster, routable from nowhere	derived token a per-share credential computed from one root key by HMAC, so nothing has to store or fan out 3000 of them	project id · volume id the labels a share is addressed by through the gateway, instead of by its Service name
Recreate the Deployment strategy that stops the old pod before it starts the new one	RWO attach fence the CSI attachment that makes two hub pods on one PVC impossible, not merely unlikely	nfs.nfs4_unique_id the client-side kernel parameter that gives a node a client identity a rename survives	capture · generation a mutation-complete record of what changed, and the closed unit a flush publishes

Data flow

Every file operation in the pod is an S3 request. Nothing is buffered, flint writes nothing of its own into the bucket — and the privilege that makes the mount is not in the tenant's namespace.



THE MOUNT HAPPENS BEFORE THE MOUNTER RUNS, and that is the whole trick. The node plugin opens /dev/fuse and calls mount(2) itself — the one act that genuinely needs privilege, done once, per node — then hands the descriptor to an unprivileged worker pod over SCM_RIGHTS, where an unchanged mount-s3 serves it needing no privilege at all. The tenant pod is given nothing: no sidecar, no label, no webhook, no credential, no privileged container. It declares a csi: volume naming a CR in its own namespace and stays admissible under PodSecurity restricted.

THERE IS NO CONTROLLER AND NO STATUS, because nothing about a passthrough mount converges. The CRD is the whole control surface and the node plugin reads it directly; what a (cluster issuer, namespace, ServiceAccount, CR) tuple is actually GIVEN is decided by the broker's backend — static for a rig, sts for a proxy that speaks AssumeRoleWithWebIdentity, rest for an application's own JWT-enforcing API. A CR on a cluster the project's policy never named yields nothing.

A UI NEEDS NOTHING BUILT, which is the one place passthrough is the cheapest of the four. The bucket IS the API: a UI lists and GETs the same keys the mount presents, so there is no gateway to run, no inbox to drain and no manifest to respect. What it costs is the same thing the mount costs — no coordination. Two S3 clients with no lease between them can overwrite each other, and Mountpoint's own limits still bind what a UI can usefully do: no rename, no append, no in-place modification, so "edit a file" is always a whole-object PUT.

EVERY OPERATION IS A REQUEST, so there is no recovery point to state: nothing is buffered and nothing is flushed. That is also the cost — the S3 round trip is in the pod's critical path, where lean has local disk and lite has one wire to one process. The bucket, meanwhile, is untouched by flint: this is the only front end you can point at a prefix someone else's tooling owns.

A WORKER IS NEVER TAKEN AWAY FROM A TENANT STILL USING ITS MOUNT — by ORDERING, not by a PodDisruptionBudget. A budget only applies to the eviction API, and on a spot fleet the ordinary case is kubelet's graceful shutdown, where the eviction API is not involved at all. What is built instead is a PriorityClass that ranks a worker above its tenants, so kubelet tears it down last, and a preStop hook that waits for NodeUnpublishVolume to release the volume.

→ data plane — every file operation, intercepted

→ the object path — the same operation, as a request

→ control plane — never carries a file

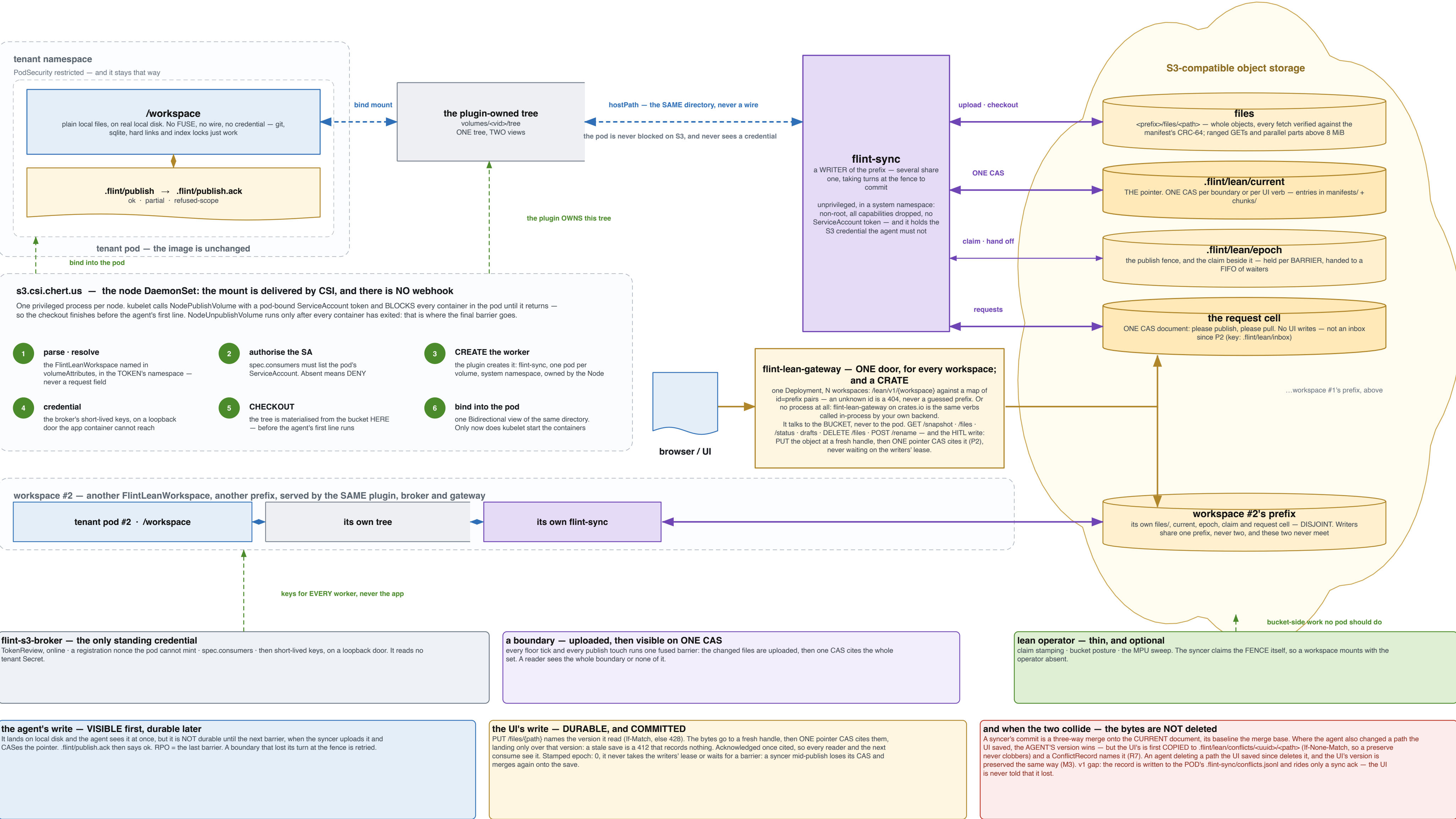
→ privilege — the one act that needs it, once per node

What the abbreviations mean

FUSE filesystem in userspace: the kernel hands each file operation to a process instead of a disk	/dev/fuse the descriptor that IS the mount. Whoever holds it serves the filesystem	mount(2) the syscall that attaches a filesystem to a path — the one act here that genuinely needs privilege	SCM_RIGHTS the Unix-socket message that passes an open file descriptor from one process to another
CSI ephemeral inline a volume declared in the pod spec itself — no PVC, no PV, nothing to bind beforehand	requiresRepublish kubelet re-calling NodePublishVolume every 60-90 s so the credential can be refreshed	TokenReview the Kubernetes API that says whether a ServiceAccount token is still valid — online, not offline	registration nonce a binding the node plugin makes for this pod and CR, which the pod itself cannot mint
spec.consumers the ServiceAccounts a mount may be used by. ABSENT MEANS DENY, never means everyone	loopback door AWS_CONTAINER_CREDENTIALS_FULL_URI: how short-lived keys reach the worker, and only the worker	SigV4 the signature an S3 wire accepts. No S3 endpoint takes a bearer token, which is why keys exist at all	PodSecurity restricted the strictest built-in pod policy. A csi: volume is on its allow-list; a privileged container is not
ENOTCONN what every open file returns once the mounter is gone. Running containers are stranded, not retried	Bidirectional the mount propagation that lets a mount made on the host appear inside the pod	FlintPassthroughMount the custom resource one mount is declared as, in the tenant's own namespace	PriorityClass · preStop how a worker outlives its tenant on a drain or a spot reclaim — ordering, not a budget

Data flow

Nothing sits between the pod and its files. A worker beside it — holding the credential the pod must not — reconciles the same directory with the bucket at a boundary the agent declares.



THE ORDERING IS THE ARCHITECTURE. kubelet blocks every container in the pod on NodePublishVolume, so the checkout completes before the agent's first line runs — no init container, no readiness dance — and calls NodeUnpublishVolume only after every container has exited, which is where the final barrier goes. In between, the app touches plain files on real local disk with zero interception.

THE MOUNT IS DELIVERED BY CSI, AND THERE IS NO WEBHOOK. The tenant pod declares a csi: volume naming a FlintLeanWorkspace in its own namespace and stays admissible under PodSecurity restricted: no injected sidecar, no mutating admission, no cert bootstrap, no privileged container and no S3 credential anywhere in the tenant namespace. Privilege is concentrated instead — the node plugin (one per node, holding no S3 credential and no Secrets RBAC), the worker (non-root, no ServiceAccount token) and the broker (the only standing credential, every issuance TokenReview-verified and audit-logged).

THE BOUNDARY VERBS ARE FILES, because the workspace is the interface. An agent that can write a file can declare a coherent point — echo > .flint/publish — and learn when its bytes are durable, from .flint/publish.ack: ok means the bytes are in S3, partial names what a foreign write kept out, and a boundary that lost its turn at the fence is retried rather than refused. No client library, no credential, no network path.

A UI IS POWERED BY THE BUCKET, NOT BY THE POD, and that is forced rather than chosen: the workspace tree is local disk that dies with the pod, and the pod may not exist at all, so there is nothing for a UI to call. flint-lean-gateway — opt-in, its own bearer, an explicit workspace map, and deliberately NOT the lite gateway — reads and writes the same CAS cells the syncer uses, and since 0.1.0 it is a CRATE too: a backend depends on flint-lean-gateway and calls the verbs in-process, holding nothing but credentials for the prefixes it serves. GET /snapshot returns {manifest, manifest_etag, inbox} in one read — the field keeps the old name and carries only the request cell's two standing requests; GET /files/<path> reads the handle the manifest cites, and since every UI edit commits, that is the newest version and a read is one fetch; delete and rename are ONE pointer CAS each (a delete stops citing the path and deletes no object, its tombstone naming what it retired; a rename moves the citation, no bytes move); drafts are durable edits nobody sees until promoted.

THE HITL WRITE IS TWO ORDERED STEPS AND THE SECOND IS THE COMMIT (P2, since 2026-09-25). PUT /files/<path> writes the OBJECT at a fresh handle first, then ONE pointer CAS cites it, judged against the version the UI read: a save over a version someone else published since is a 412 that records nothing, and the UI re-reads and reconciles. It never takes the writers' lease and never waits for a barrier — a syncer mid-publish loses its CAS, merges again onto the save and retries — so the cost of heavy saving falls on the writers, never on the person saving. Before step 5 a write was an object plus an inbox entry a syncer later cited; that cell now carries only the two verb requests.

A DEPOSED WORKER CANNOT WRITE, AND NOTHING CAN WEDGE THE UI. The worker-facing verb (POST /manifest) is epoch-validated PER REQUEST, so a write whose claimed epoch is not the cell's current epoch is rejected — rotation alone leaves that door open, which is exactly what the model's LeanNoEpochCheck mutation proves. And there is no barrier window any more: a UI save never waits on a worker, so a dead one blocks nobody (Prop_UISaveCompletes, fair to the gateway's CAS alone).

AND ONE VERB IS DELIBERATELY CARRIED, NEVER PERFORMED. "Please publish" from outside the pod is honoured; "please pull" is recorded and left to the agent. A boundary publishes what is already on disk and touches no local file, while a sync re-derives the tree and DELETES local files for remotely-deleted paths — so performing it on a remote's say-so would upgrade what a leaked gateway bearer can do from "publish, plus hand over these N named objects" to "rewrite and delete across a running agent's tree, at my timing, under a scope I choose". v1's recorded limits: whole-object HITL writes under a cap, and one shared bearer.

NOTHING HERE IS INJECTED, AND THERE IS NO WEBHOOK IN ANY OF IT. The flint-sync worker is CREATED by the node plugin during NodePublishVolume — one pod per published volume, in a system namespace, pinned with nodeName so it skips the scheduler, and owned by the Node object so a vanished node garbage-collects it. It is not a container in the tenant's pod and no mutating admission rewrites the tenant's spec: the component is the SYNCER, and flint-sync is its binary. The gateway, the broker and the operator are not injected either — each is an ordinary Deployment installed by Helm, and the gateway is opt-in: its chart REFUSES to render without a token Secret and a workspace map, because an unauthenticated gateway is an open writer to every workspace configured.

WHY THE GATEWAY MAY CAS THE MANIFEST AFTER ALL. The old reason it did not still holds: a BOUNDARY is a claim about a tree, and the gateway has no tree (its root is literally /nonexistent), no baseline and no scan. P2 makes that not matter. A UI verb edits ONE path's citation, judged against the version the UI read, and claims nothing about any other path. And a writer no longer adopts a pointer that moved under it: its baseline IS its merge base, what its tree is owed is derived from the document at each consume (P1-lite), and its commit is a three-way merge onto the CURRENT document — so a save that landed between a syncer's read and its CAS is merged in, never lost.

CONCURRENT UI WRITERS ARE SETTLED BY THE ONE CAS. Every overwrite NAMES what it read: 428 precondition-required without an If-Match, 412 file-changed on a stale one, and the current etag on the 412's own header — forge's taxonomy, adopted so the three doors are one shape. Two browsers that each read v1 and then save: one CAS lands, the other is a 412 that records nothing and re-reads. A verb that loses its CAS to the WRITERS rather than to another save re-reads, re-judges and retries. Before If-Match was required, two such browsers BOTH succeeded — the second silently winning.

AND WHY NOT SIMPLY ROUTE TO THE WORKER, the way lite's gateway routes to a hub? Because half the time there is nothing to route TO. A lean worker is created by the node plugin at NodePublishVolume and DELETED at NodeUnpublish after the drain, so a workspace no tenant pod is mounting has no flint-sync anywhere — it is only a prefix in a bucket, and a UI must still be able to list and write it. A lite hub is a Deployment an operator can scale 0→1 and wake; a lean worker is a side effect of a POD being scheduled, and no gateway can conjure a tenant pod. Nor is there a door to route to: flint-sync's control surface is a UDS in the pod's emptyDir, and the code says why it stays that way — "there is no TCP listener and no authentication, because the trust boundary is the pod; a TCP listener would be a new remote surface, which is a stated non-goal".

AND ROUTING WOULD HAND THE UI THE WRONG GUARANTEE, which is the part that surprises. A write placed into the worker's tree is visible at once and durable only at the next barrier — the AGENT's bargain. The UI's 200 would then precede the bytes reaching S3, and a pod that dies in between loses the write. The gateway's commit inverts exactly that: durable and committed when the call returns. The same line is already drawn inside the pod — the UDS sync verb EXECUTES, "because the caller is inside the pod: it is the agent asking for its own tree to be updated, which is the agent's own decision to make", while the gateway's sync request is carried and never performed. Inside the pod is a decision; outside it is a proposal.

WHAT P2 COSTS, AND WHERE IT FALLS. Every UI verb moves the pointer every syncer and reader checks, so an editor autosaves to a DRAFT, never with a commit: drafts live under the reserved namespace no scan, checkout or sweep can see, and a promote commits conditioned on the base it recorded. A syncer whose CAS loses to a save merges again, so heavy saving costs the writers. And what a save replaced is not deleted: nothing cites it, the retire log keeps it for the retire age G (600 s) and only then may a sweep take it, so a reader that loaded the document less than G ago can still fetch every handle it cites (Inv_ReaderFetches).

THE GATEWAY IS A WRITER, AND THE TWO DOORS STILL MEAN DIFFERENT THINGS: the agent's write is visible before it is durable, the UI's is durable and committed when the call returns. The gateway stamps epoch: 0 and never takes the fence — its CAS is judged against the version the UI read, not against the lease — while a syncer holds the fence for its own commit. So "written" still means different things at the two doors, and a UI that reports success on a PUT is reporting a commit.

TWO WORKSPACES SHARE THE MACHINERY AND NOTHING ELSE. The node plugin is one per published volume, the broker is one Deployment, and the gateway is one process whose workspace map is a list of id=prefix pairs — an unknown id is a 404, never a guessed prefix, and in v1 ONE bearer covers every workspace in that map, which is the recorded limit to know before pointing a UI at a multi-tenant one. Everything below that is per volume: its own worker pod (one per PUBLISHED volume, created by the plugin on its own node and owned by the Node so a vanished node GCs it), its own tree, its own credential, and its own prefix with its own epoch, claim, pointer and request cell. Taking turns at one prefix's fence is a mechanism inside a product; across products it is only a convention, so what assigns prefixes is what keeps two workspaces apart.

THREE QUESTIONS DECIDE AGAINST LEAN, cheapest disqualifier first: does the tree fit the disk, and its file count the checkout budget (the manifest is chunked and a publish is O(changed), but every checkout still materialises every file); is a shared log with per-pod working copies enough (a same-file edit is the later boundary's, the other kept); is snapshot freshness acceptable. A no on any of them means the hub. And a worker is never taken away from a tenant still using its tree by ORDERING, not by a PodDisruptionBudget — a PriorityClass for kubelet's graceful shutdown and a preStop that waits for the release, because on a spot fleet the eviction API is not involved at all.

-----> the workspace — one directory, two views, never a wire

-----> durable path — the writers and the gateway, a CAS at a time

-----> control plane — never carries a file

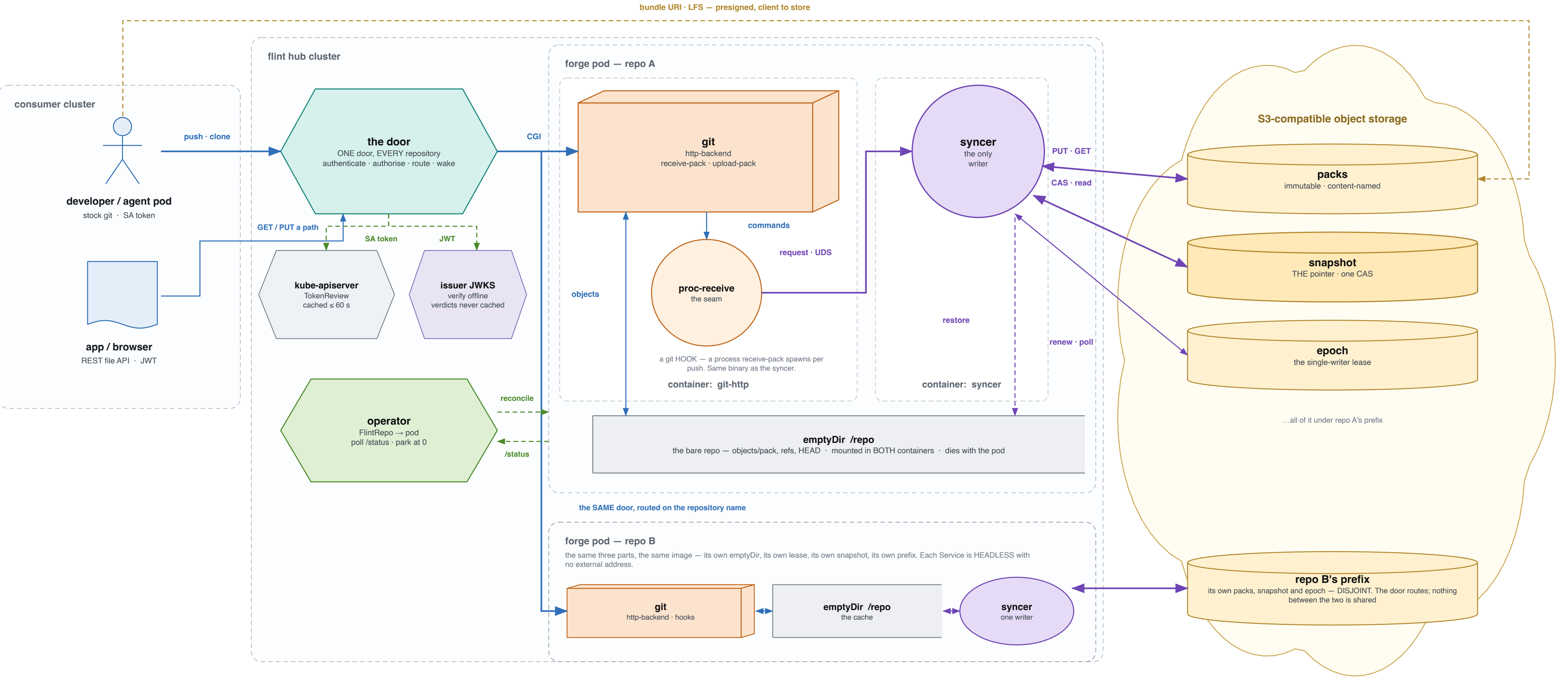
-----> the file protocol — boundary verbs in the tree, REST at the gateway

What the abbreviations mean

CSI ephemeral inline a volume declared in the pod spec itself — no PVC, no PV, nothing to bind beforehand	NodePublishVolume the CSI call kubelet BLOCKS every container on. The checkout happens inside it	NodeUnpublishVolume the call kubelet makes only after every container has exited — the final barrier	bind mount · hostPath two names for ONE directory: the plugin's tree, seen from the pod and from the worker
CAS compare-and-swap — a write that lands only if the object is still the version you read	ETag / If-Match S3's precondition: the ETag you read, offered back as the terms of the write	the pointer .flint/lean/current — the ONE mutable metadata object. Entries live in immutable manifests	fence whose turn it is to commit: held for one barrier's commit, then handed on. A deposed holder's late CAS is refused
epoch · lease the bucket-side commit cell: one holder, a FIFO of waiters. It lives in the BUCKET, so it fences a writer in ANY cluster	claim the project identity stamped on the prefix: adopt your own, refuse a foreign one	boundary verb a file the agent writes to declare a coherent point, and an ack file it reads back	boundary one fused barrier: upload the changed files, then install the whole set with one CAS
loopback door AWS_CONTAINER_CREDENTIALS_FULL_URI: how short-lived keys reach the worker, and only the worker	TokenReview the Kubernetes API that says whether a ServiceAccount token is still valid — online, not offline	FlintLeanWorkspace the custom resource one lean project is declared as, in the tenant's own namespace	RPO recovery point. For lean it is the last BARRIER, never the last write
HITL human in the loop: a write that reaches the workspace from a UI rather than from the agent	the request cell the two verb requests from outside the pod, please publish and please pull. Not an inbox any more: its key keeps the old name, .flint/lean/inbox	P2 since 2026-09-25 a UI verb COMMITS: one pointer CAS, judged against the version the UI read, else 412	epoch-validated checked PER REQUEST against the cell's current epoch, so a deposed worker's write is refused

Data flow

Stock git serves every byte. ONE stateless door fronts every repository; one syncer beside each is the only process that can write S3.



ONE DOOR, MANY REPOSITORIES — and this is exactly where forge and flint-lite differ. The door holds no repository state: it verifies the token, resolves the repository named in the URL to its FlintRepo, routes to that repository's pod and wakes it if it is parked — so a thousand repositories are ONE endpoint, one DNS name and one credential shape to every client. A lite hub cannot be fronted that way, because the hub IS the tree: one process, one prefix, dialled directly at the address its own Service advertises. A second lite volume is a second pod, a second Service and a second mount at the consumer. And the door is not only a router: where the operator renders the NetworkPolicy, it is the ONLY peer admitted to a repository pod's git port — which is what makes X-Remote-User mean anything. Where that policy is NOT rendered, reaching the port IS the authorisation — and the operator raises PrincipalsUnenforced against that repository rather than leaving it unsaid. Either way a client that bypasses the door is already outside the trust model, so the door is a COMPLETE oracle of who pushed and when — which is what the idle ladder counts. A lite hub has no such place.

TWO CREDENTIAL KINDS, SERVED AT ONCE. The door routes on the token's iss: a pod's ServiceAccount token goes to TokenReview at the apiserver, an issuer-minted JWT to offline signature verification against the cached JWKS. A JWT's sub — a person — becomes the principal for spec.consumers, for the branch policy, and as the commit author.

AND THE DOOR REFUSES TO START if the cluster's ServiceAccount issuer and the configured JWT issuer are the same string: every pod token would then route to the offline verifier, which cannot know a pod was deleted.

ONE CAS ON THE SNAPSHOT is the transaction boundary. Told “ok” means that object moved; a 412 on it is a fence, not a retry.

What the abbreviations mean

- CAS**
compare-and-swap — a write that lands only if the object is still the version you read
- UDS**
Unix domain socket — the hook-to-syncer channel, inside the pod, never over a network
- SA token**
a Kubernetes ServiceAccount token, projected into the pod and rotated by the kubelet
- iss / sub / exp**
a JWT's issuer, its subject — the principal — and its expiry

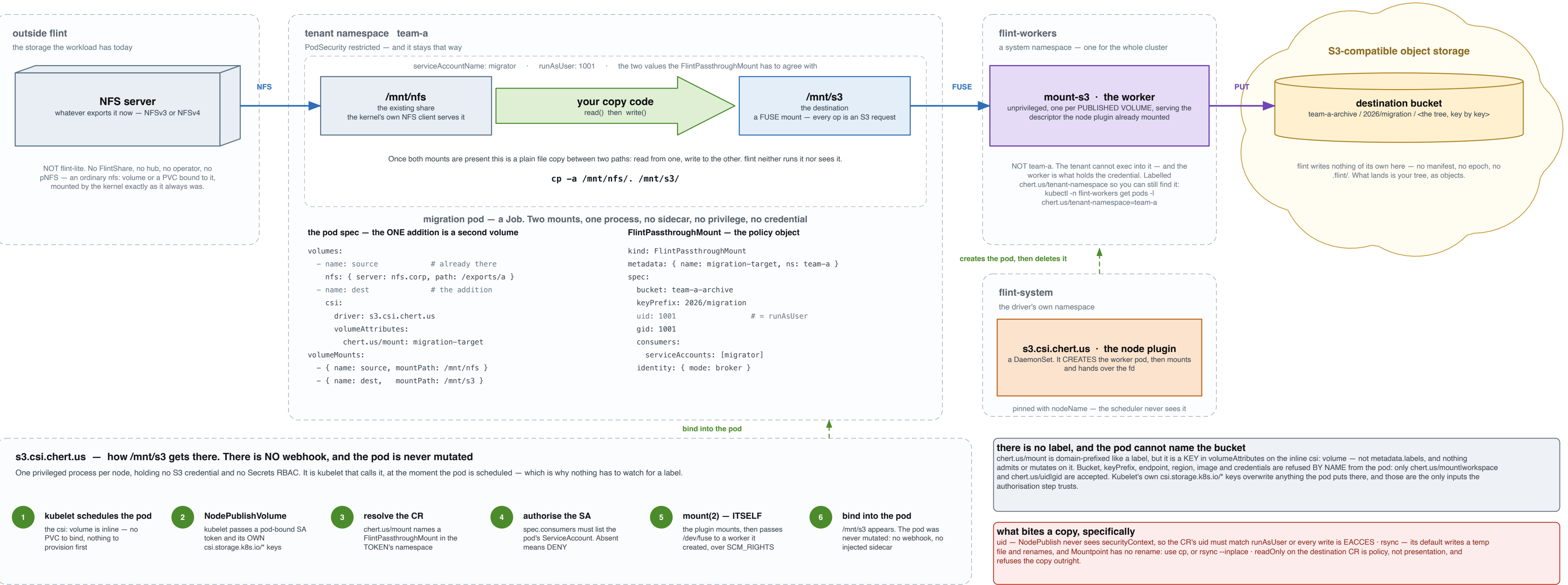
- ETag / If-Match**
S3's precondition: the ETag you read, offered back as the terms of the write
- proc-receive**
the git hook that lets the server, not git, decide each ref update
- TokenReview**
the Kubernetes API that says whether a ServiceAccount token is still valid
- LFS**
Git Large File Storage — big blobs kept beside the repository, moved by presigned URL

- fence**
a lost CAS. Another server holds this repository, so this one stops READING too, and exits
- pre-receive**
the git hook that applies the branch policy, and can refuse the whole push
- JWT**
JSON Web Token — a signed bearer token, verified at the door with no round trip
- bundle URI**
git's own opt-in: the server names a pack the client fetches straight from S3

- CGI**
Common Gateway Interface — how the runner invokes git http-backend, once per request
- emptyDir**
a Kubernetes volume that lives and dies with the POD — not with a container
- JWKS**
JSON Web Key Set — the issuer's signing keys. The KEYS are cached, the verdicts are not
- FlintRepo**
the custom resource that one forge repository is declared as

Migration — an NFS share into a bucket

The pod already mounts its NFS share. A SECOND volume in the same pod spec names a FlintPassthroughMount, and the CSI node plugin mounts the destination bucket at /mnt/s3. The copy itself is ordinary code in the pod — flint moves no bytes of its own.



THE POD IS CREATED THE ORDINARY WAY, and that is the whole point of the shape. You apply a Job whose spec has two volumes: the nfs: one it already had, and a csi: one naming a FlintPassthroughMount. There is no mutating webhook in flint at all, so nothing rewrites the pod between kubectl and kubelet — what you applied is what runs. The mount appears because kubelet calls NodePublishVolume on the node plugin when it schedules the pod, and the plugin does the one privileged act itself. A typo in chert.us/mount is therefore not an admission error but a FailedMount event naming the CR and the namespace, and the pod simply never starts.

IT USED TO BE A LABEL, and anyone who set this up before v1.45.0 will look for one. Until then a mutating webhook watched for flint.io/passthrough-mount on the pod — objectSelector, key Exists — and the label's VALUE named the CR, exactly as the volumeAttributes value does now. What it injected was a PRIVILEGED native sidecar, and that privilege was not reducible: /dev/fuse alone would take CAP_SYS_ADMIN and a hostPath device, but the mount has to REACH the app container, which needs mountPropagation: Bidirectional, which the API server permits only on a privileged container. So a namespace enforcing PodSecurity baseline or restricted REJECTED the mutated pod — correctly. 'fcac038f' replaced both sidecar-injection webhooks (passthrough's and lean's) with the node DaemonSet, because spec.volumes[*].csi is on the restricted allow-list. The information you supply did not change; its carrier did.

THE WORKER IS A POD, AND IT IS NOT IN YOUR NAMESPACE. It lives in flint-workers — one system namespace for the whole cluster, not one per tenant — created by the node plugin during NodePublishVolume, pinned to this node with nodeName so the scheduler never places it, and owned by the Node object so a vanished node garbage-collects it. Three reasons it is not in team-a: the worker is what holds the credential, and anyone who can exec into pods in their own namespace could read it; PodSecurity is enforced differently, since flint-workers must be labelled privileged (the lean tree needs hostPath, forbidden under baseline) while your namespace stays restricted; and the plugin's own RBAC is a Role in flint-workers ALONE — pods create/get/list/watch/delete there and nowhere else, with a ValidatingAdmissionPolicy that further pins spec.nodeName to the node making the call, so a compromised node agent cannot place a pod anywhere but on itself. You can still find yours: chert.us/tenant-namespace and chert.us/tenant-pod are labels on the worker.

THE POD NAMES A CR; THE CR NAMES THE BUCKET. That indirection is the security property worth drawing, because it is what makes the destination a decision the platform owns rather than one the workload asserts. The pod author cannot point a mount at an arbitrary bucket by editing their own manifest — volumeAttributes are attacker-controlled input to a privileged process, and every key but the selector and the two integers is refused by name. Whoever may create a FlintPassthroughMount in that namespace decides which buckets exist to be named, and spec.consumers decides which ServiceAccounts may name them.

THE COPY IS YOUR CODE, running between two paths that happen to be backed by very different things: /mnt/nfs is served by the kernel's own NFS client and /mnt/s3 by a FUSE mount whose every operation is an S3 request. Nothing coordinates them and nothing needs to — the process reads a file and writes a file. What that costs is the S3 round trip in the copy's critical path, so throughput comes from concurrency: several files in flight, or several pods each given a subtree. What it saves is everything else — no staging disk, no intermediate format, no second copy of the data.

A RE-RUN IS SAFE, because a key is idempotent: writing the same object again yields the same object. That makes a Job with a backoff the right shape and makes “did it finish?” answerable by listing both sides rather than by trusting an exit code — the destination is a bucket, so the census is a LIST and the comparison is name-and-size. What a re-run will NOT do is repair a partially written object in place: Mountpoint writes each file once, sequentially, as a whole-object PUT, so an interrupted file is re-written from the beginning rather than resumed.

WHEN THIS IS THE WRONG TOOL. A passthrough mount is not POSIX — no rename, no append, no in-place modification — so it suits a copy and suits nothing that wants to keep editing afterwards. If the workload's own writes must continue against the destination with ordinary file semantics, the destination wants flint-lean (a local tree with a durable path behind it) or flint-lite (an NFS server whose working set is a PVC). Migrating INTO one of those is a different picture: the destination mount is theirs, and the copy step is the same plain code.

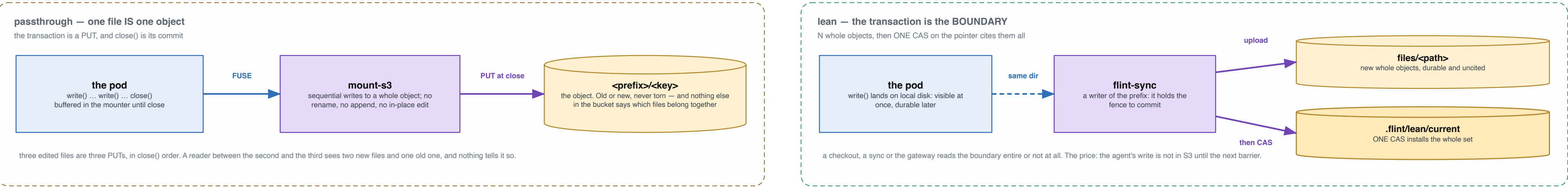
→ data plane — the bytes the copy moves → the object path — the same bytes, as requests - - - - - → control plane — how the mount arrives

What the abbreviations mean

CSI ephemeral inline a volume declared in the pod spec itself — no PVC, no PV, nothing to bind beforehand	volumeAttributes the map on that inline volume. It is where chert.us/mount goes — NOT metadata.labels	NodePublishVolume the CSI call kubelet makes when it schedules the pod. This is when the mount happens	FlintPassthroughMount the CR that names the bucket, prefix, uid and consumers. The POLICY object
chert.us/mount the only key that selects a destination. It names a CR in the pod's own namespace, never a bucket	spec.consumers the ServiceAccounts a mount may be used by. ABSENT MEANS DENY, never means everyone	mount(2) the syscall that attaches a filesystem to a path — the one act here that genuinely needs privilege	SCM_RIGHTS the Unix-socket message that passes an open file descriptor from one process to another
FUSE filesystem in userspace: the kernel hands each file operation to a process instead of a disk	mount-s3 Mountpoint for S3, unchanged upstream. It serves the descriptor the node plugin already mounted	kernel NFS client what serves /mnt/nfs. In-tree, nothing to do with flint — the share is mounted as it always was	flint-s3-broker the one Deployment holding a standing credential. It issues short-lived keys to the worker only
EACCES on every write what a uid mismatch looks like. The CR's uid must match the pod's runAsUser	no rename Mountpoint cannot rename. rsync's default temp-file-then-rename fails; cp and rsync --inplace do not	whole-object PUT each file is written once, sequentially. There is no append and no in-place modification	Job, not Deployment a copy that finishes should be a Job — and a re-run is safe, because keys are idempotent

ACID — passthrough vs lean

Both doors put whole objects in a bucket. Passthrough keeps ONE ledger and inherits S3's guarantees exactly; lean keeps TWO — the objects and a manifest that cites them — and pays for every property it adds with a recovery point on the agent's writes.



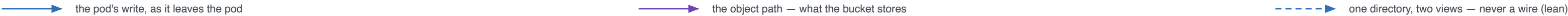
flint-passthrough — one ledger: the bucket		flint-lean — two ledgers: the objects, and the manifest that cites them	
A — atomic does a reader ever see half of a write? And what is the unit — one file, or a set?	PER OBJECT, YES — and that is the whole unit. A file becomes an object when it is closed; until then nothing is in S3 and no reader sees it. S3's PUT is all-or-nothing, so a reader sees the previous object or the new one, never a torn one. ACROSS FILES, NO: three edited files are three independent PUTs landing in close() order, and a reader between them sees one new and two old. No rename, no append and no in-place modification at any setting, so "edit" is always a whole-object replace — and a directory move is N copies.	PER BOUNDARY — N files, one CAS. Every changed file is uploaded as a whole new object, then ONE CAS on .flint/lean/current cites the whole set: a checkout, a sync or the gateway reads the boundary entire or not at all. A rename is one generation — destination entry and source removal in the same CAS. A UI's save, delete or rename is ONE CAS of its own, cited when it is acknowledged. The one reader with no atomicity is a raw-key reader on files/: a browser, or a passthrough mount on the same prefix, sees objects land one by one.	
C — consistent what invariant holds after every write, and who checks it?	S3's, UNMEDIATED. Read-after-write per object: a completed PUT is visible to every reader's next GET. No invariant spans two objects, and flint adds none — no manifest, no checksum of its own, no claim on the prefix; the bucket is left exactly as other tooling finds it. What the MOUNT adds is staleness: a metadata TTL (Minimal unless --cache is set; a CR may pass its own) during which a listing or a stat can be old, and no data cache unless --cache is set, so a re-read is a re-GET.	MANUFACTURED by the manifest, and CHECKED on every read. A generation cites, per path, the object's ETag and a CRC-64 the writer computed; every fetch is verified against it and a mismatch refuses the checkout with nothing written. The claim refuses a foreign prefix; the epoch fences a deposited writer on every request. The invariants are machine-checked against the shape that ships (LeanP1.tla): an acknowledged UI write is never lost unaccounted (Inv_AckedNamed), and a handle is never cited under two names (Inv_OneName).	
I — isolated can two writers, or a writer and a reader, corrupt each other?	NONE, BY DESIGN. Two pods on one prefix do not see each other's intent: two writers of one key are last-close-wins, undetected — no lock, no lease, no conditional PUT. Two readers can each see a different mix of generations across files. A reader never sees a writer's partial object, because it is not in S3 until close. The only lever is readOnly, per mount. A pod that wants git, pip or sqlite on the tree wants lean, whose boundary is what makes those safe.	ONE COMMIT AT A TIME, ordered by the pointer CAS. Writers take turns at the fence (the epoch cell) for their commit only; a CAS on a stale read is refused and merged three ways onto the current document, and a deposited holder is fenced on every request. A UI save is one CAS judged against the version it read (428 / 412) and never waits for the fence; a draft is invisible until promoted. When the agent and a UI change the same path, the agent's version wins and the UI's is preserved as a conflict copy, never deleted.	
D — durable when the call returns, where are the bytes — and what can still lose them?	AT close(). write() buffers in the mounter; close() or fsync completes the upload and returns only when S3 holds the object, so a returned close() is durable in S3 and there is no recovery point to state — nothing else is buffered. The same fact from the other side: an open file whose mounter dies is lost whole, nothing is on local disk, and every read is a request in the pod's critical path.	AT THE BARRIER. write() returns when local disk has the bytes — visible at once, durable later. The syncer uploads at cadence (floorSecs) or when the agent writes .flint/publish, and the ack says ok only when the boundary is in the bucket. RPO is the last barrier, never the last write: a pod that dies between barriers loses the difference (a graceful shutdown drains at NodeUnpublish). A checkout fsyncs every file before it writes the marker that vouches for it; a UI's PUT is durable and committed when it returns.	
what TIES — per object, both are S3 untorn, read-after-write, strongly consistent since 2020-12. A single-file edit through either door is equally safe; the difference begins with the second file, and with the question of who else may be writing.		what NEITHER gives multi-writer coherence, byte-range locks, cross-pod O_EXCL — those need an arbiter that answers in milliseconds, which is the hub. And neither is isolated from a raw-key writer holding the credential: lean DETECTS the foreign overwrite (every fetch is If-Match the cited etag: refused, not served); passthrough cannot.	

TWO LEDGERS IS THE WHOLE DIFFERENCE. Passthrough writes objects and nothing else, so the bucket is the transaction log and S3's per-object guarantees are the entire contract — which is exactly why a passthrough mount can be pointed at a prefix somebody else's tooling owns and leave it as it was. Lean writes objects AND a manifest that cites them, and the CAS on the manifest is what makes N objects one transaction, gives every fetch a checksum to verify against, and lets a lease say who may write. Every one of those is bought with the barrier: the agent's write is visible on local disk at once and in S3 only at the next boundary.

THE UI'S WRITE IS A THIRD CONTRACT, and it is the agent's turned around. Through the gateway — or the flint-lean-gateway crate called in-process — a write is DURABLE AND COMMITTED when the call returns: the object at a fresh handle, then ONE pointer CAS judged against the version the UI read (P2, since 2026-09-25). The agent's write is VISIBLE when write() returns and DURABLE at the barrier. The pointer CAS is what keeps the two safe together: a syncer whose CAS loses to a save merges again onto it, and every gateway read is the citation, one fetch.

“CONSISTENT” MEANS SOMETHING DIFFERENT AT EACH DOOR. At passthrough it is S3's word: read-after-write on one key. At lean it is the database's word: an invariant that holds across the whole tree after every transaction — every cited object exists at the cited ETag with the cited CRC, no path is under two names, and no acked write is lost. The first is inherited; the second is manufactured, and it is manufactured by the same manifest that costs the recovery point.

READ THE DURABILITY ROW WITH ITS COST. Passthrough's close() is the strongest single-file durability on the page — the bytes are in S3 before the call returns — and it is paid for on every read and every write with a round trip in the pod's critical path. Lean's local-disk ack is the weakest, and it is what lets git, sqlite and a build cache run at local speed; the boundary verbs exist so the agent can buy durability exactly when it wants it, for exactly the files it names.



What the abbreviations mean

ACID atomic, consistent, isolated, durable: the four questions a database answers about one transaction	the transaction the unit the four questions are asked of. For passthrough, ONE object; for lean, ONE boundary	close() where a passthrough file becomes an object: the mounter completes the upload on close or fsync, and not before	metadata TTL how long the mount trusts a listing or a stat before asking S3 again — Minimal unless --cache is set
CAS compare-and-swap — a write that lands only if the object is still the version you read	ETag / If-Match S3's precondition: the ETag you read, offered back as the terms of the write	boundary a coherent point: the set of files one manifest generation cites, installed by ONE CAS on the pointer	the pointer .flint/lean/current — the ONE mutable metadata object. Entries live in immutable, chunked manifests
epoch · fence the bucket-side commit cell: one writer commits at a time, and a deposited holder is refused	P2 since 2026-09-25 a UI verb COMMITS: one pointer CAS, judged against the version the UI read, else 412	CRC-64 the checksum the manifest carries per entry, computed by the client that moved the bytes; every fetch is checked	RPO recovery point. Passthrough has none to state; lean's is the last BARRIER, never the last write

The protocol — one shared log, many working copies

Lean is a strongly consistent shared log with eventually consistent working copies: git with automatic push and pull. One conditional write on the bucket orders every boundary; each writer's TICK keeps its tree close to that log, and a UI reads the bucket and commits to it directly, one CAS per verb.

